

4-bit Ripple UP Counter

Introduction:

A 4-bit ripple up counter is a sequential digital circuit that counts from 0 to 15 (binary 0000 to 1111) using T (Toggle) flip-flops connected in a ripple configuration. In this design, each T flip-flop toggles its output when its clock input receives a falling edge. The counter is called "ripple" because the toggle effect ripples through the flip-flops sequentially, with each flip-flop using the inverted output of the previous stage as its clock input. This creates an asynchronous counting mechanism where changes propagate through the counter like ripples in water.

Code Description:

The Verilog code shows 4-bit ripple up counter implemented using four T flip-flops (TFF0, TFF1, TFF2, TFF3). The main clock signal is connected directly to TFF0, while subsequent flip-flops receive their clock signals from the inverted outputs of the previous stages. All flip-flops share a common asynchronous reset signal that initializes the counter to 0000. The outputs q_out0, q_out1, q_out2, and q_out3 represent the LSB to MSB respectively, forming the 4-bit output count.

Block Diagram:

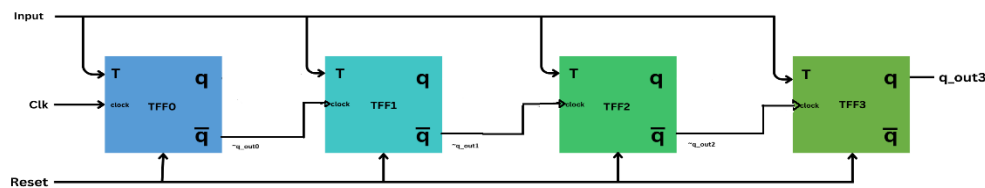


Fig: 4-bit ripple up counter

Truth Table:

Clock Pulse	Q_out3 (MSB)	Q_out2	Q_out1	Q_out0(LSB)	Binary	Decimal
0	0	0	0	0	0000	0
1	0	0	0	1	0001	1
2	0	0	1	0	0010	2

Clock Pulse	Q_out3 (MSB)	Q_out2	Q_out1	Q_out0(LSB)	Binary	Decimal
3	0	0	1	1	0011	3
4	0	1	0	0	0100	4
5	0	1	0	1	0101	5
6	0	1	1	0	0110	6
7	0	1	1	1	0111	7
8	1	0	0	0	1000	8
9	1	0	0	1	1001	9
10	1	0	1	0	1010	10
11	1	0	1	1	1011	11
12	1	1	0	0	1100	12
13	1	1	0	1	1101	13
14	1	1	1	0	1110	14
15	1	1	1	1	1111	15
16	0	0	0	0	0000	0(Repeat)

Application:

Ripple counters are widely used in digital systems for applications such as frequency division, digital clocks, event counting, time delay generation, and as building blocks for more complex sequential circuits. They are particularly useful when simplicity of design is more important than speed, as they require minimal logic gates but have propagation delay limitations.

Procedure to create a lab:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir ripple_up_counter`
- 1.4: Open project directory: `cd ripple_up_counter`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim ripple_up_counter_4bit.v`

RTL Design Code:

// T-Flipflop

```
module tff (  
    input clk,  
    input reset,  
    output reg q  
);  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            q <= 1'b0;  
        else  
            q <= ~q;  
        end  
    endmodule
```

// Ripple up counter code

```
module ripple_up_counter_4bit (  
    input clk,  
    input reset,  
    output [3:0] q  
);  
    wire q_out0, q_out1, q_out2, q_out3;  
    assign q = {q_out3, q_out2, q_out1, q_out0};  
    // TFF instantiation  
    tff tff0 (.clk(clk), .reset(reset), .q(q_out0));  
    tff tff1 (.clk(~q_out0), .reset(reset), .q(q_out1));  
    tff tff2 (.clk(~q_out1), .reset(reset), .q(q_out2));  
    tff tff3 (.clk(~q_out2), .reset(reset), .q(q_out3));  
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Create Testbench file

3.1: Open gvim editor for testbench with command:

```
gvim ripple_up_counter_4bit_tb.v
```

Testbench:

```
module ripple_up_counter_tb;
    reg clk;
    reg reset;
    wire [3:0] q;
    // 4-bit ripple up counter instantiation
    ripple_up_counter_4bit uut (
        .clk(clk),
        .reset(reset),
        .q(q)
    );
    // Clock generation
    always #5 clk = ~clk;
    initial begin
        clk = 0;
        reset = 1;
        // Display signals
        $display("Time\treset\tq");
        $monitor("%0dns\t%b\t%b", $time, reset, q);
        // Apply reset
        #10;
        reset = 0;
        #200;
        $finish;
    end
    // Waveform dump

    initial begin
        $dumpfile("up_counter_dump.vcd");
        $dumpvars();
    end
endmodule
```

3.2: Save the testbench file using command- :wq!

3.3: Check the saved file in current directory: use command ls

Step 4: Simulate the Design with Verilator

4.1: Use the following command to simulate:

```
verilator --binary -j 0 -Wall ripple_up_counter_4bit.v\  
ripple_up_counter_4bit_tb.v --top ripple_up_counter_tb\  
--timing --CFLAGS "-std=c++20" --trace
```

4.2: If any error is available, resolve it.

4.3: After simulation a new directory obj_dir will be created by tool, check it by using command `ls`.

4.4: Open obj_dir: `cd obj_dir`

4.5: Use command `make -f Vripple_up_counter_tb.mk
Vripple_up_counter_tb`

4.6: A new directory with name `Vripple_up_counter_tb` will be created

4.7: Use command `./Vripple_up_counter_tb` to compile

4.8: Check output at terminal

Simulation output:

The simulation output shows the behavior of a 4-bit ripple up counter implemented using T flip-flops. Initially, when `reset=1`, all flip-flops are cleared and the output `q` shows 0000. Once reset is deasserted (`reset=0`), the counter begins counting upward on each positive edge of the clock. The counter increments from 0000 to 1111 (decimal 0 to 15) and then wraps around back to 0000, demonstrating the expected ripple counter behavior. Each T flip-flop toggles when its clock input transitions from high to low, creating the characteristic ripple effect through the counter stages.

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file:

`gtkwave up_counter_dump.vcd`

5.2: Analyze the waveform to verify the correct operation of the 4-bit ripple up counter



Output Waveform:

The output waveform demonstrates the ripple up counter's operation with clear visualization of the asynchronous reset functionality and the sequential counting behavior. When reset is high, all outputs are forced to 0. After reset is released, the counter begins incrementing with each clock pulse. The waveform shows the characteristic ripple delay between stages, where each flip-flop toggles based on the inverted output of the previous stage, creating the binary counting sequence from 0000 to 1111 and then repeating. This confirms the proper functionality of the T flip-flop based ripple up counter design.